Lập trình mạng · Chương 4

# RPC: lời gọi thủ tục từ xa và Java RMI

TS. Đặng Ngọc Hùng

# Chương 4: Kỹ thuật lập trình phân tán

## 1 RPC

- Lời gọi thủ tục từ xa và Java RMI
- Truyền tham số, idempotency và truy cập đồng thời trong RMI

## 2 Web service

- SOAP, hợp đồng WSDL và message XML
- REST, tài nguyên, phương thức HTTP và JSON

## 3 gRPC

- Protocol Buffers và lời gọi unary
- Streaming, deadline và mã trạng thái

## 4 Message broker

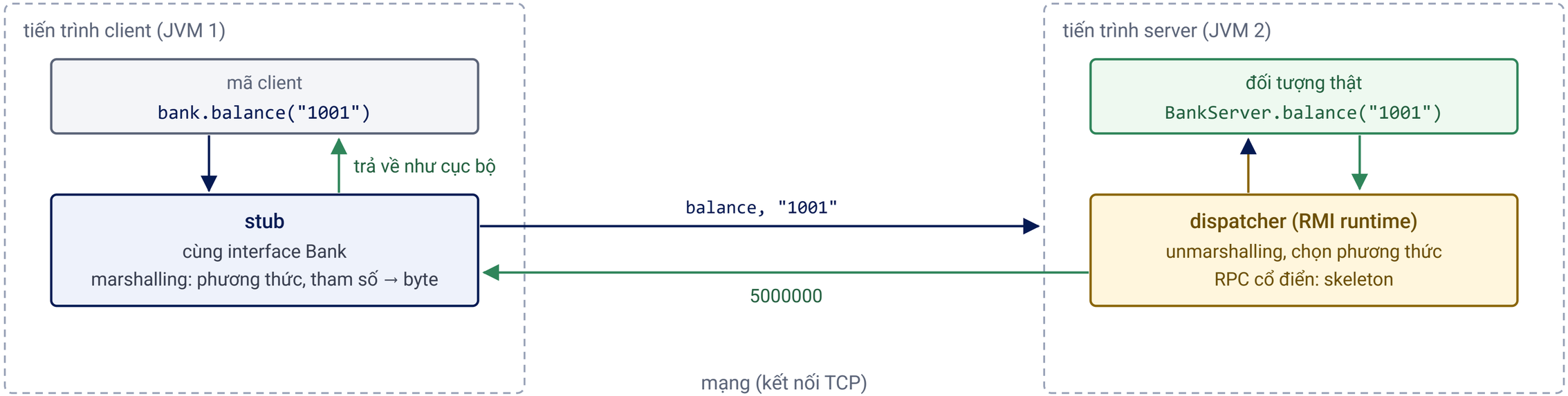
- RabbitMQ, exchange, queue và binding
- Bài toán ứng dụng: xử lý đơn hàng qua message broker

# Lời gọi từ xa: gọi phương thức trên máy khác như gọi cục bộ

1. Ba chương trước: tự định nghĩa giao thức, tự **đọc ghi** từng dòng, từng datagram
2. **RPC**: gọi phương thức trên máy khác như gọi cục bộ; thư viện đóng gói tham số và truyền qua mạng
3. Lời gọi vẫn đi qua mạng: có thể **thất bại** theo cách lời gọi cục bộ không có

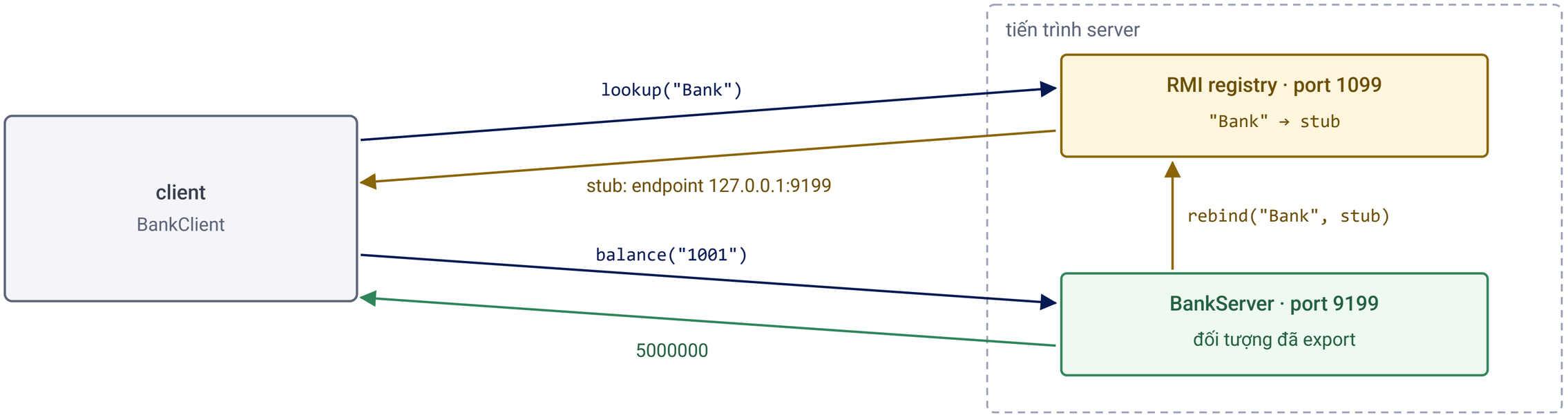
> **Câu hỏi của phần:** lời gọi từ xa hoạt động thế nào, viết bằng Java RMI ra sao, và thất bại thế nào?

# Stub đóng gói lời gọi, dispatcher ở server gọi phương thức thật




**Stub** phía client và **dispatcher** phía server lo **marshalling** và truyền; mã ở hai đầu chỉ thấy một lời gọi phương thức.

# Client tra tên trong RMI registry để lấy stub, rồi gọi thẳng đối tượng




**RMI registry** chỉ ánh xạ tên tới stub; lời gọi đi thẳng từ stub tới endpoint của đối tượng.

# Remote interface là hợp đồng chung của client và server

```java
public interface Bank extends Remote {

    /** Số dư (đồng) của tài khoản. */
    long balance(String accountNo) throws RemoteException, AccountNotFoundException;
}
```

- **Remote interface**: kế thừa `java.rmi.Remote`; server hiện thực, client gọi qua stub
- Mọi phương thức khai báo **RemoteException**: lời gọi có thể thất bại vì mạng hoặc vì server
- Lỗi nghiệp vụ là ngoại lệ riêng; tham số, giá trị trả về đi qua mạng bằng **serialization**

# Server export đối tượng, đăng ký stub; RMI runtime nhận lời gọi

Java

```
System.setProperty("java.rmi.server.hostname", "127.0.0.1");  // địa chỉ ghi vào stub
bank = new BankServer(delay);
Bank stub = (Bank) UnicastRemoteObject.exportObject(bank, 9199);  // nhận lời gọi ở port 9199
registry = LocateRegistry.createRegistry(1099);  // RMI registry chạy trong tiến trình server
registry.rebind("Bank", stub);  // đăng ký stub dưới tên "Bank"
```

Kết quả chạy

```
bound "Bank" in RMI registry on port 1099; object on port 9199
balance("1001") on thread RMI TCP Connection(2)-127.0.0.1
balance("9999") on thread RMI TCP Connection(2)-127.0.0.1
```

# Client tra tên, nhận stub và gọi như gọi một đối tượng cục bộ

Java

```
Registry registry = LocateRegistry.getRegistry("127.0.0.1", 1099);  // stub của registry, chưa kết nối
Bank bank = (Bank) registry.lookup("Bank");  // tra tên, nhận stub của ngân hàng
System.out.println("stub class: " + bank.getClass().getName());
RemoteObject handler = (RemoteObject) Proxy.getInvocationHandler(bank);  // stub là dynamic proxy
System.out.println("stub ref: " + handler.getRef().remoteToString());
long amount = bank.balance(accountNo);  // lời gọi từ xa, cú pháp như lời gọi cục bộ
```

Kết quả chạy

```
stub class: jdk.proxy1.$Proxy0
stub ref: UnicastRef [liveRef: [endpoint:[127.0.0.1:9199](remote),objID:[-27061e4c:1a17302a670:-7fff, -6559209415889670914]]]
balance(1001) = 5000000
balance(9999) -> no such account: 9999
```

# Lời gọi thất bại giữa chừng: client không biết server đã chạy hay chưa

Java

```
Thread.ofPlatform().start(() -> {
    sleep(500);
    server.destroyForcibly();  // tắt hẳn server khi balance đang chạy
});
call(bank, 1);
server.waitFor();
call(bank, 2);
```

Kết quả chạy

```
  [server] bound "Bank" in RMI registry on port 1099; object on port 9199
  [server] balance("1001") on thread RMI TCP Connection(2)-127.0.0.1
server process killed
call 1 after 511 ms: java.rmi.UnmarshalException
  cause: java.net.SocketException: Connection reset
call 2 after 2 ms: java.rmi.ConnectException
  cause: java.net.ConnectException: Connection refused: connect
```

- Lần gọi 1: **UnmarshalException** sau khoảng 500 ms, trong khi server đã bắt đầu chạy `balance`
- Lần gọi 2: **ConnectException** ngay lập tức: không còn tiến trình nào nhận ở port 9199
- Lời gọi từ xa thất bại: client **không biết** server đã thực hiện phương thức hay chưa

# Câu hỏi 1

## Trong Java RMI, stub nằm ở phía nào và làm việc gì?

A. Ở server; nhận byte từ mạng và gọi phương thức của đối tượng thật.

B. Ở client; có cùng interface với dịch vụ, đóng gói lời gọi và gửi tới endpoint của đối tượng.

C. Ở RMI registry; chuyển tiếp mọi lời gọi từ client tới server.

D. Ở client; giữ bản sao dữ liệu của server để trả lời mà không cần mạng.

# Câu hỏi 1

## Trong Java RMI, stub nằm ở phía nào và làm việc gì?

A. Ở server; nhận byte từ mạng và gọi phương thức của đối tượng thật.

B. Ở client; có cùng interface với dịch vụ, đóng gói lời gọi và gửi tới endpoint của đối tượng.

C. Ở RMI registry; chuyển tiếp mọi lời gọi từ client tới server.

D. Ở client; giữ bản sao dữ liệu của server để trả lời mà không cần mạng.

Stub là đối tượng phía client có cùng interface với dịch vụ; nó đóng gói lời gọi và gửi tới endpoint của đối tượng. Nhận byte và gọi phương thức thật là việc của dispatcher ở server; registry chỉ ánh xạ tên tới stub.

# Câu hỏi 2

## Client gọi một phương thức RMI và nhận UnmarshalException trong lúc chờ kết quả. Có thể kết luận gì về phía server?

A. Server chưa nhận lời gọi, nên gọi lại luôn an toàn.

B. Server đã thực hiện xong phương thức.

C. Không biết chắc: server có thể chưa chạy, đang chạy dở hoặc đã chạy xong phương thức.

D. RMI sẽ tự gọi lại cho tới khi thành công, nên client không cần xử lý.

# Câu hỏi 2

## Client gọi một phương thức RMI và nhận UnmarshalException trong lúc chờ kết quả. Có thể kết luận gì về phía server?

A. Server chưa nhận lời gọi, nên gọi lại luôn an toàn.

B. Server đã thực hiện xong phương thức.

**C. Không biết chắc: server có thể chưa chạy, đang chạy dở hoặc đã chạy xong phương thức.**

D. RMI sẽ tự gọi lại cho tới khi thành công, nên client không cần xử lý.

Lỗi xảy ra sau khi lời gọi đã gửi đi, nên phía client không phân biệt được server đã thực hiện phương thức hay chưa. RMI không tự gọi lại.

# Tổng kết

1. **RPC**: gọi phương thức trên máy khác như gọi cục bộ; **stub** và dispatcher lo **marshalling** và truyền
2. Java RMI: **remote interface** kế thừa `Remote`; server export đối tượng, đăng ký stub vào **RMI registry**
3. Mọi phương thức khai báo **RemoteException**; khi lỗi, client không biết server đã thực hiện hay chưa

**Từ khóa cần nhớ** RPC · IPC · stub · marshalling · remote interface · RMI registry · RemoteException

# Mở rộng

### Chủ đề nâng cao

- Các ngộ nhận về hệ phân tán (fallacies of distributed computing)
- Vì sao RMI ít dùng giữa các hệ thống viết bằng ngôn ngữ khác nhau

### Bài tập mở rộng

Thêm phương thức balances(List<String> accountNos) vào ngân hàng RMI; so sánh thời gian một lời gọi với nhiều lời gọi balance.

Tiếp theo

# RPC: truyền tham số, idempotency và truy cập đồng thời trong RMI